Types of Architectures

Monolithic Architecture – when we just start building an application

Website vs Web Applications → Read + (Write Actions) eg fb → Read Info
→ Update status
→ Upload Info.

(Dynamic content

→ which have static content → read only application
(can't update anything)

eg
- Wikipedia
- Newspaper websites
- Blogging

Web Applications →

Three layers

① Frontend / UI → Representation layer.

↓ call

② Backend / Business → Data – Info / Image etc →

Someone has to create that data →

Contains all logic needed to express that data

↓ call

③ Data storage

Response

Skill of web application → Diff^t options of building app.

Choose option → Most Optimal Application →
Detailed Implementation level Analysis.

Architecture → Internal design details for building the applications

↓

Monolithic Architecture Pattern

↓

a single

→ All 3 layers of web app] → coded together / deployed together] same language

Disadvantage :

1. In monolithic architecture, every module is combined in a single system, so if there is an error or bug in a single module, it can destroy the complete system. 2. In monolithic architecture, whenever a single module is updated, the whole system needs to be updated to reflect the changes to the users. All modules are present in a single system and are connected to one another, so the whole system needs to be updated. 3. In monolithic architecture, if there is any change in a single module's programming language or framework, it will affect the entire system. The entire system needs to be changed because every module is interlinked and tightly coupled.

When to use? → (Advantage)

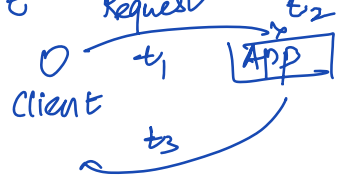
→ when you're just started → Can't go wrong

→ when we've latency sensitive app. e.g.

↓

Can't afford >> Δt

[Since every component is in a single m/c – no need n/w calls (which is very slow)]



$\Delta t = t_1 + t_2 + t_3$ (latency)

→ Its comparatively easy to secure.

→ integration testing is comparatively easy — because of less moving pieces.

→ less confused developers. {all are comfortable}

Any other architectures → mostly are not aware.

Option 1: Monolithic architecture requires fewer resources since it has a single code repository, a single code file, single code base and is also easier to manage. A monolithic application is built as a single unit. Cross-cutting concerns are the concerns that affect the whole application such as logging, handling, caching, and performance monitoring. In a monolithic application, this area of the functionality concerns only one application so it is easier to handle it. Option 2: Monolithic systems should be preferred by developers with little or no experience because: Simple to develop. As long as the monolithic approach is a standard way of building applications, any engineering team has the right knowledge and capabilities to develop a monolithic application. Simple to deploy. Another advantage associated with the simplicity of monolithic apps is easier deployment. When it comes to monolithic applications, you do not have to handle many deployments – just one file or directory. Easier debugging and testing. In contrast to the microservices architecture, monolithic applications are much easier to debug and test. Since a monolithic app is a single indivisible unit, you can run end-to-end testing much faster. Less cross-cutting concerns. Cross-cutting concerns are the concerns that affect the whole application such as logging, handling, caching, and performance monitoring. In a monolithic application, this area of the functionality concerns only one application so it is easier to handle it. Option 3: Monolithic systems support only limited users since it's difficult to scale this kind of architecture. We cannot scale components independently, only the whole application can be scaled in this architecture. It is harder to implement changes in such a large and complex application with highly tight coupling. Any code change affects the whole system so it has to be thoroughly coordinated. This makes the overall development process much longer. Also, when a monolithic application scales up, it becomes too complicated to understand. Also, a complex system of code within one application is hard to manage. Option 4: This is incorrect as mentioned in the above option that monolithic applications can only support limited users.

MICRO SERVICES

- multiple smaller applications instead of monolithic architecture
  - → Single Responsibility
  - → Loose Coupling

App 1 [Service Requester] → App 2 [Service Provider]

Each application should be independent of each other.

↓

have own database

(communication → API calls)

Design checkout experience

Cart service → Checkout service → Inventory service → Shipment service

Checkout service → Payment service

Checkout service → Authentication service

Advantages

1. Selective scaling is easy
   (eg if we just want to scale backend
2. Loose coupling — (assume contract doesn't change → change in one doesn't affect change in other)
3. Smaller deployment time
4. free to use multiple tech. styles
5. Proper ownership & Better Collaboration

Disadvantages

1. ↑ cost [separate m/c s etc]
2. Many dynamic pieces → Tricky Integrity Testing → might miss some perm. & comb.
3. Security : expensive & complex
4. Higher latency : Additional n/w ops.

Monolithic	Microservices
→ Together ↓ low Latency	→ Granularity ↓ ↑ Latency
→ Security : Easy	→ Complex | Expensive
→ can't use multiple tech stack	→ ✓
→ Selective scaling ✗	→ ✓
→ Tightly coupled	→ loosely coupled ↳ Easy to use.